

板凳龙螺线运动模型及其路径优化

摘要

“板凳龙”作为我国浙闽地区传统民俗活动，在元宵时节备受人们青睐。在舞龙队演出过程中，每个人在行进中的速度与位置都至关重要，并且龙头前进和调头路线的选择也会直接影响盘龙面积的大小进而影响到节目效果的整体观感。本文针对舞龙队沿着等距螺旋线盘入的情形建立几何模型，设计了合理高效的算法，研究了龙队行进过程的位置与速度、螺线间距、调头路线、盘入与盘出曲线之间的关联，对优化板凳龙观赏性有一定作用。

对于问题一，本文建立了**阿基米德螺旋线的几何模型与龙头运动模型**，提出了**把手位置递推算法与速度传递算法**，逐步推导计算出龙身各把手的位置与速度。首先，在计算各点位置时考虑到直接联立方程的复杂性，本文采用**拟牛顿法**得出了方程的解，确定了龙身各把手位置随时间的变化关系。其次，根据同一个板凳上沿着板凳长度方向的速度相等的物理规律并计算各点在螺线上的切线方向，**递推计算**出每个点的速度。

对于问题二，在问题一的基础上，本文首先建立了**碰撞检测模型**，其次证明了有关碰撞的预备理论：(1) 相比于龙身，龙头更容易发生碰撞。(2) 相比于等距螺旋线的外圈，内圈更容易发生碰撞。碰撞检测模型的建立为仿真过程中判断整条龙内是否发生碰撞提供了依据，结合证明的碰撞预备理论，在很大程度上**简化了求解过程**。然后利用仿真模型不断逼近，得到龙内结构首次发生碰撞时刻(**412.2763s**)以及该时刻舞龙队的位置和速度。

对于问题三，本文利用问题一的**阿基米德螺旋线模型**与问题二的**碰撞检测模型**，进行**仿真实验**确定最小螺距。将龙头固定于圆形调头空间的边缘向外盘开，在不发生碰撞的条件下，缩小螺线间距直至临界条件（恰好不发生碰撞）。找出螺线最小值后对结果进行检验，检验舞龙队是否从起点盘入调头空间都未发生碰撞，否则重新寻找最小值。考虑到螺线间距越小，越容易发生碰撞，本文采用**二分法**的算法得到螺距最小值为 **D = 45.032665cm**。

对于问题四，本文提出了**板凳龙全行进路线几何模型**。同时在问题一的基础上进一步完善了复杂曲线情况下的**龙头运动模型**、**把手位置传递算法与速度传递算法**，从而在全路线中使用。问题四的目标是对板凳龙完整行进路线进行优化，以减小调头曲线长度。在 S 形曲线起点的极角 θ_s （极坐标系下）变化的过程中，部分把手点出现坐标突变、速率呈负值（即回退）的情况，由此限定了 θ_s 的下界。以 θ_s 为决策变量，在限定范围内建立调头曲线长度函数，该长度与 θ_s 呈负相关，计算得调头曲线的理论最小长度为 **13.621m**，此时 S 形曲线起点恰好位于调头区域边界圆上，即**不能**使调头曲线变短。利用**把手位置传递算法与中心差分法**可以求得调头前后各把手位置与速度。

对于问题五，本文建立了舞龙队**速度比值计算模型**，基于问题二中**速度传递算法**的思想，同一板凳龙结构下各把手速度呈定比，结合问题四得到的舞龙队轨迹，适当给出舞龙开始时刻及结束时刻，并对该时间区间以极小时间步长进行细分，得到所有时刻最大把手速度与龙首把手速度比值，最后根据该比值得到龙首把手速度满足题目条件的最大值(**v = 1.2463m/s**)。

关键词：等距螺旋线 递推 二分法 碰撞检测 仿真实验 中心差分法

一、问题重述

1.1 问题背景

为了较好的观赏效果，板凳龙演出时要在在保证龙队自由盘入和盘出的情况下追求更小的盘龙面积和更快的盘龙速度，因此需要建立数学模型确定舞龙队每个人在规定盘入曲线和盘出曲线上位置、速度随时间的变化和盘龙时螺线之间的距离。舞龙队由 233 节板凳组成，其中第一节为龙头，最后一节为龙尾，其他各节均为龙身。龙头由长为 341cm ，宽为 30cm 的板凳构成，龙身和龙尾均由长为 220cm ，宽为 30cm 的板凳构成。每节板凳上均有两个孔，孔径为 5.5 cm ，孔的中心距离最近的板头 27.5 cm ，相邻两条板凳通过把手连接。

1.2 需要解决的问题

1. 当等距螺线间距为 55cm 、龙头从螺线第 16 圈 A 点处开始盘入、龙头前把手速度为 1m/s 时：
 - (1) 建立合适的几何模型。
 - (2) 根据几何模型，求解从初始时刻到 300s 每秒整个龙队的位置和速度并填表。
2. 当等距螺线间距为 55cm 、龙头从螺线第 16 圈 A 点处开始盘入、龙头前把手速度为 1m/s 时：
 - (1) 建立碰撞检测模型，求解恰好不发生碰撞的时刻。
 - (2) 求解该时刻龙队的位置与速度。
3. 当调头空间是以螺线中心为圆心，直径为 9m 的圆且龙队能盘入到圆的边界时：求解等距螺旋线的最小螺距。
4. 当龙头前把手速度为 1m/s ，盘入曲线的螺距为 1.7m ，调头空间圆的直径为 9m ，盘出螺线与盘入螺线关于螺线中心呈中心对称，调头路径的两段圆弧半径比为 $2:1$ ，圆弧与盘入曲线、盘出曲线均相切，两段圆弧之间也相切时：
 - (1) 求解调头曲线的最小值。
 - (2) 求解从 -100s 到 100s （以调头时刻为 0 时刻）每秒龙队的位置与速度并填表。
5. 当龙头匀速行进，行进路线与问题四相同，舞龙队各把手的速度不超过 2m/s 时：求解龙头行进的最大速度。

二、问题分析

2.1 问题 1 的分析

龙队内部每个点（本文将龙队把手视为串连在一起的点，将长板凳视为连接在一起的杆，下同）所在位置、运动方向都不一样，无法直接求出每个点的速度，但在已知第一个点的运动情况下，可以采用递推的方法确定后方所有点的运动情况。每个点的位置可通过联立解析式求得而速度的求解还要结合“同一杆上的两点沿杆方向速度相同”这一物理规律，由于等距螺旋线在直角坐标系下无简洁表达式而直线方程在极坐标下也不够简单，可以采用拟牛顿法计算方程的解。

2.2 问题 2 的分析

该问的关键点是如何判断龙内各杆间是否发生碰撞并且如何在盘龙时检测碰撞。

根据分离轴定律可以判断两个矩形是否发生碰撞，但是由于龙内的矩形板凳众多并且每个点的位置速度都不同，无法使用计算的方法对碰撞进行检测，因此本文采用仿真实验的方式找出了龙队内部恰好不发生碰撞的临界时刻，而每个点位置与速度的计算仍然采用问题 1 的方法。为了减少繁杂冗余的非必要检测，还需要寻找并证明减少碰撞检测的预备理论。

2.3 问题 3 的分析

本题的变量为螺线间距，意在找出满足目标的最小变量。根据题意可以将龙头固定于圆形边缘向外盘开，通过改变螺线间距确定最小螺线值使得龙队在该位置时不发生碰撞。将所得结果进行检验，检验其是否从圆形边缘回退到圆形边缘的次外圈均不发生碰撞。若检验通过则可确定螺线最小值，检验不通过则重新寻找满足题意的螺线最小值，下次检验时将龙头置于本次碰撞点开始回退。螺距的下限为 0，上限为 55cm，为了快速找到目标最小值，可以采用二分法寻找。

2.4 问题 4 的分析

本问要求在保持 S 形曲线各处相切的情况下，减小调头曲线长度，同时给出调头前后部分把手位置与速度。为探求 S 形调头曲线长度影响因素，需要在问题三所构建盘入几何模型的基础上，增加盘出螺线与 S 形曲线模型，从而构建板凳龙全行进路线几何模型。利用该模型，可以通过平面几何方法求得 S 形曲线长度函数，并利用计算机仿真、碰撞检测算法探求 S 形曲线减小的潜在制约因素。为计算调头前后特定把手位置与速度，需扩展第一问的把手位置递推算法与速度传递算法。

2.5 问题 5 的分析

本问要求在问题四所建立模型下求取龙头的最大行进速度。本问为最优化问题，龙头速度是决策变量也同时是目标函数，各把手速度均小于 2m/s 为约束条件，要求在约束条件下最大化龙头速度。为减小计算量，需结合速度传递算法中对各把手速度特征的分析，并利用贪心算法思想与微分法求出特定龙头速度下的全过程最大速度，使得目标函数达到最优解。

三、模型假设

1. 不考虑舞龙队真实盘入、盘出螺线与标准等距螺旋线之间的误差。
2. 板凳龙在行进时足够平稳，不考虑板凳晃动、上下颠簸、左右摇摆。
3. 板凳质量分布均匀，板凳连接处无非人为因素引起的旋转或摩擦。
4. 不考虑人为对突发情况的应急措施，如碰撞前的减速。
5. 调头空间内舞龙队能够继续沿螺线前进并选择合适时机走 S 形曲线，调头空间内与盘入螺线相连的螺线仍属于盘入螺线，与盘出螺线相连的螺线仍属于盘出螺线，调头路径只包含圆弧线，调头曲线包含调头空间内的所有曲线。
6. 整个龙队只有第一个把手提供动力，其他把手只是受到牵引被动地在曲线上前进。

7.调头空间内 S 形曲线与盘出、盘入螺线的切点也关于原点中心对称。

四、符号说明

符号	说明	单位
v_i	龙队中第 <i>i</i> 个点的速度	m/s
α_i	第 <i>i</i> 个点的切线与该点连接 的前一个杆的夹角	Rad
l_0	龙头板凳的长度	m
l_1	龙身板凳的长度	m
$S(\theta)$	阿基米德螺旋线的长度	m
t	从初始时刻开始经过的时间	s
(ρ_i, θ_i)	第 <i>i</i> 个点的极坐标	(m,rad)
t_0	龙队盘龙步进时的单位时间	s
d_{max}	等距螺线的螺距上限	m
d_{min}	等距螺线的螺距下限	m
d	螺线间距	m

五、模型的建立与求解

5.1 问题 1 的模型建立与求解

本问要求盘入过程中各把手的位置与速度，由假设 6 可知龙身各把手不具有动力，因此需要先构建龙头运动模型，再利用几何关系建立把手位置递推算法与速度递推算法，求得各把手位置与速度。

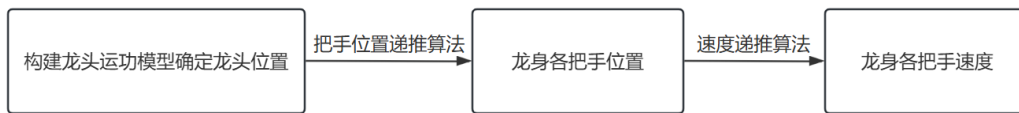


图 0: 问题 1 求解流程图

5.1.1 螺旋形几何模型与板凳龙简化模型的建立

本问只涉及求解每个点的位置和速度，不考虑板凳之间的碰撞，因此可以合理忽略板凳的宽度，在建立几何模型的过程中只保留每个板凳的长度，当舞龙队进入等距螺旋线后，几何模型如图 1 所示：

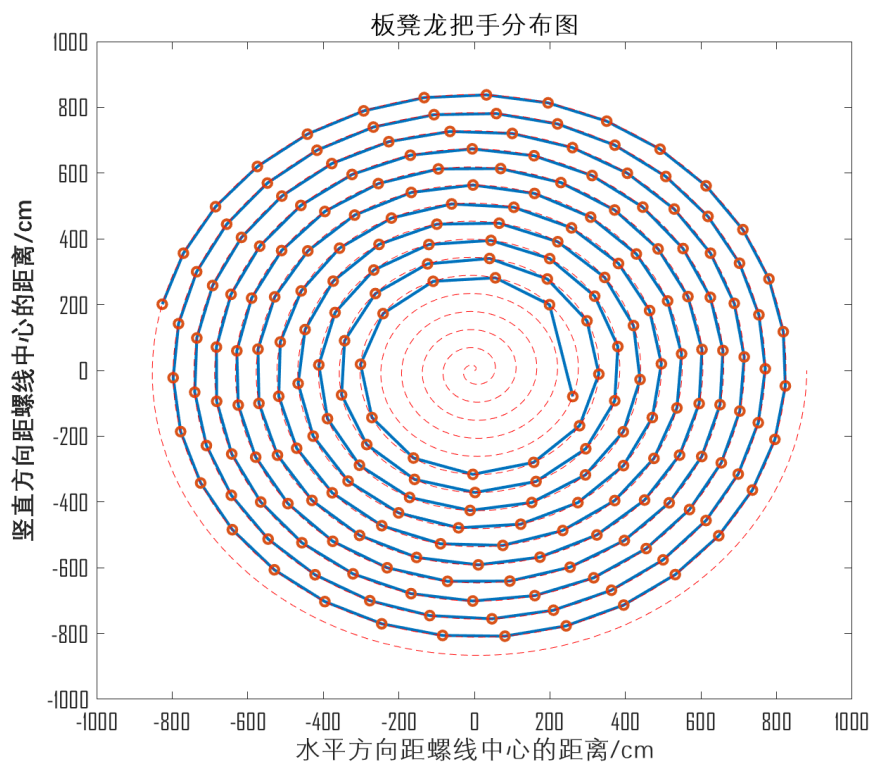


图 1: 板凳龙把手分布图

5.1.2 龙头运动模型的构建

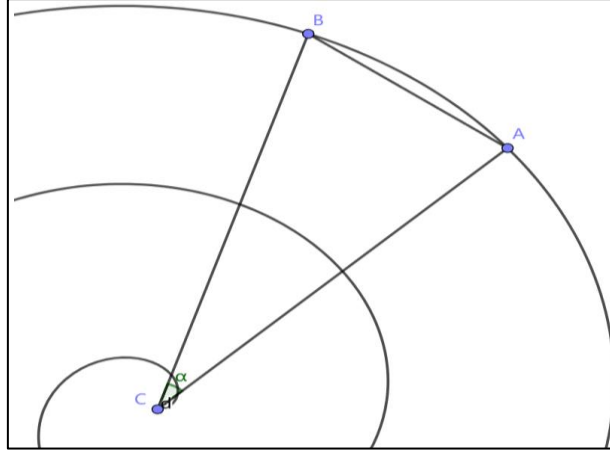


图 2：龙头运动及把手位置传递演示图

假设图 2 中 A 为龙头把手，整个龙队在 A 点带领下向螺线内盘入， $v_1 \equiv 1m/s$ 。

不难算出点 A 走过的弧长 $l = v_1 t$ 。查阅相关文献可知，阿基米德螺旋线的长度 $S(\theta) =$

$\frac{1}{2}a[\theta\sqrt{1+\theta^2} + \ln(\theta + \sqrt{1+\theta^2})]$ ，其中 a 为螺线间距。构建方程计算出 A 点坐标：

$$\begin{cases} v_1 t = S(16\pi) - S(\theta_1) \\ \rho_1 = a\theta_1 \end{cases}$$

利用 MATLAB 编程可以求得该方程解，即得出任意时间下的龙头坐标。

5.1.3 把手位置递推算法的构建

如图 2，A 为龙头把手，B 为龙头后把手，其中 $a = 0.55m$ ，在三角形 ABC 中有：

$$l_0^2 = \rho_1^2 + \rho_2^2 - 2\rho_1\rho_2 \cos(\theta_1 - \theta_2) \quad (1)$$

点 B 在曲线上有：

$$\rho_2 = a\theta_2 \quad (2)$$

联立 (1) (2) 后再用二分法可求得 B 点坐标 (ρ_2, θ_2)

推广：若已知第 k 个点的位置计算第 $k+1$ 个点的位置。同理可得有方程：

$$l_1^2 = \rho_k^2 + \rho_{k+1}^2 - 2\rho_k\rho_{k+1} \cos(\theta_k - \theta_{k+1}) \quad (3)$$

$$\rho_{k+1} = a\theta_{k+1} \quad (4)$$

联立 (3) (4) 再利用二分法可得第 $k+1$ 个点的位置 $(\rho_{k+1}, \theta_{k+1})$ ，由此得到把手

位置递推算法。

5.1.4 龙身把手速度的推导：

一、 速度递推算法：

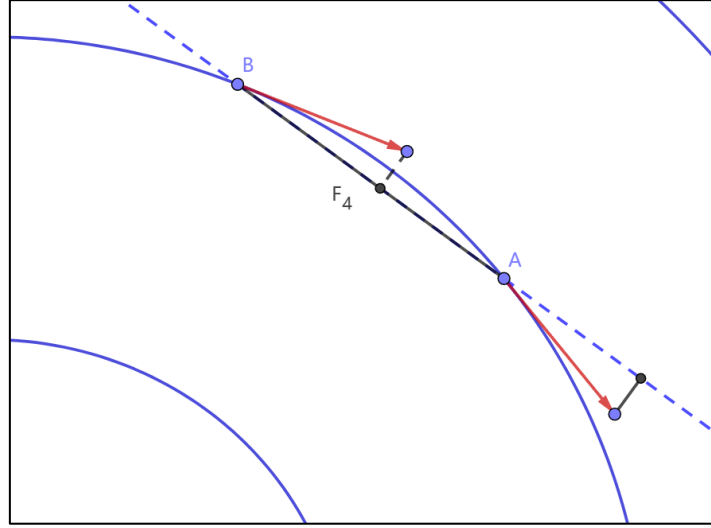


图 3：速度递推算法演示图

如图 3，A 点为前点，B 点为后点，则运动时 A、B 在直线 AB 方向上的速度大小相等。极坐标系中曲线上点的切线斜率与该点的极角有关系： $k = \frac{\sin \theta + \theta \cos \theta}{\cos \theta - \theta \sin \theta}$ ，则根据

A、B 两点的极坐标可求出两点处切线斜率 k_A 、 k_B ，直线 AB 的斜率 $k =$

$\frac{\rho_A \sin \theta_A - \rho_B \sin \theta_B}{\rho_A \cos \theta_A - \rho_B \cos \theta_B}$ ，进而可求出直线 AB 与两条切线的夹角 $\alpha_A = \tan^{-1} \frac{|k_A - k|}{1 + k k_A}$ ， $\alpha_B =$

$\tan^{-1} \frac{|k_B - k|}{1 + k k_B}$ 。根据这两点速度在 AB 直线方向上的分量相等可知：

$$v_B \cos \alpha_B = v_A \cos \alpha_A$$

解得 $v_B = \frac{v_A \cos \alpha_A}{\cos \alpha_B}$ 。

二、 中心差分法：

利用把手位置递推算法可以求得点 A 在 $t-T$ 时刻的坐标 (x_{A1}, y_{A1}) 、 $t+T$ 时刻的坐标 (x_{A2}, y_{A2}) ，当 T 足够小时，可以认为 $v_A = \frac{\sqrt{(x_{A1} - x_{A2})^2 + (y_{A1} - y_{A2})^2}}{2T}$ ，其中 t 为任意时刻， T 为时间步长。

5.1.5 龙身把手位置与速度:

在本问中，先采用龙头运动模型算出任意时刻下龙头把手的坐标，再使用把手位置递推算法计算各龙身把手位置，最后采用速度递推算法计算龙身各把手速度，计算结果见表 1、2:

龙队位置结果表

	0s	60s	120s	180s	240s	300s
龙头 x (m)	8.8000	5.7992	-4.0849	-2.9636	2.5945	4.4203
龙头 y (m)	0.0000	-5.7711	-6.3045	6.0948	-5.3567	2.3204
第 1 节龙身 x (m)	8.3640	7.4566	-1.4459	-5.2368	4.8209	2.4599
第 1 节龙身 y (m)	2.8261	-3.4408	-7.4058	4.3600	-3.5623	4.4022
第 51 节龙身 x (m)	-9.5187	-8.6862	-5.5428	2.8909	5.9797	-6.3013
第 51 节龙身 y (m)	1.3416	2.5406	6.3783	7.2491	-3.8282	0.4663
第 101 节龙身 x (m)	2.9135	5.6867	5.3615	1.8983	-4.9178	-6.2375
第 101 节龙身 y (m)	-9.9184	-8.0017	-7.5579	-8.4717	-6.3796	3.9364
第 151 节龙身 x (m)	10.8618	6.6827	2.3892	1.0056	2.9658	7.0410
第 151 节龙身 y (m)	1.8283	8.1342	9.7273	9.4247	8.3996	4.3926
第 201 节龙身 x (m)	4.5555	-6.6193	-10.6271	-9.2879	-7.4575	-7.4589
第 201 节龙身 y (m)	10.7249	9.0259	1.3603	-4.2462	-6.1804	-5.2630
龙尾（后）x (m)	-5.3059	7.3642	10.9744	7.3842	3.2415	1.7855
龙尾（后）y (m)	-10.6764	-8.7983	0.8430	7.4920	9.4692	9.3011

表 1：龙队位置结果表

龙队速度结果表

	0s	60s	120s	180s	240s	300s
龙头 (m/s)	1	1	1	1	1	1
第 1 节龙身 (m/s)	0.999971	0.999961	0.999945	0.999917	0.999859	0.99971
第 51 节龙身 (m/s)	0.999744	0.999663	0.999536	0.999331	0.998941	0.998066
第 101 节龙身 (m/s)	0.999578	0.99945	0.999266	0.998971	0.998435	0.997304
第 151 节龙身 (m/s)	0.999451	0.999298	0.999077	0.998729	0.998115	0.996864
第 201 节龙身 (m/s)	0.999351	0.999184	0.998927	0.998554	0.997894	0.996577
龙尾（后）(m/s)	0.999307	0.99914	0.998883	0.998488	0.997817	0.996482

表 2：龙队速度结果表

5.2 问题 2 的模型建立与求解

5.2.1 板凳龙简化模型的更新

本问要考虑板凳龙内部的碰撞，可能的碰撞既会发生在板凳的长度方向，又有可能发生在板凳的宽度方向，将板凳视为“杆”将会影响到实验结果，有失模型的正确性。因此从本问开始将对板凳龙简化模型进行更新，本文将它视为矩形方块，其在螺旋线路线中的模型如图 4、5 所示。

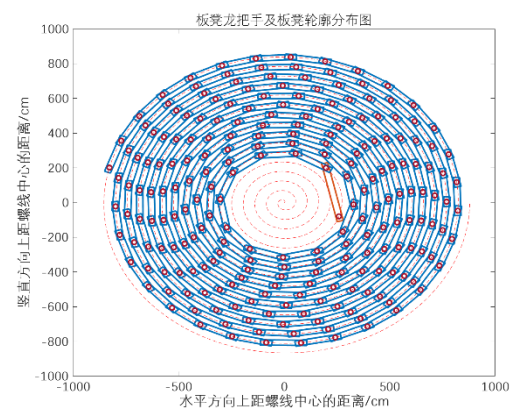


图 4：板凳龙把手及板凳轮廓分布图

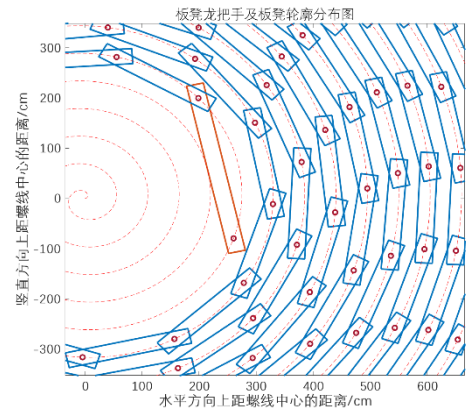


图 5：图 4 的放大

5.2.2 预备理论的证明

为简化计算，给出以下两条预备定理：（1）.相比于龙身，龙头更容易发生碰撞；（2）.相比于等距螺旋线的外圈，内圈更容易发生碰撞。为了证明以上理论，这里给出可以板凳碰撞发生概率的衡量标准，板凳的把手在沿螺旋线运动时，它的四个角将会扫过一定区域，这里称之为危险区域，当该区域无其他障碍时，将不会发生碰撞。

相比于等距螺旋线的外圈，内圈更容易发生碰撞(前把手极距较小)。

5.2.3 碰撞检测模型的建立

根据分离轴定律的思想，对于两个凸多边形物体，如果存在一个轴使得这两个凸多边形在轴上的投影不重叠，那么判定两个物体没有碰撞，否则判定为发生碰撞，本文也采取了这种投影的方法，经过多次的尝试发现所有可能的轴只会出现在与两个凸多边形各边垂直的方向上。在板凳龙运动的过程中，会出现如下图这种情形，在第一个问的基础上，我们知道了每个点的位置，不难求出两个矩形八个顶点的位置坐标。

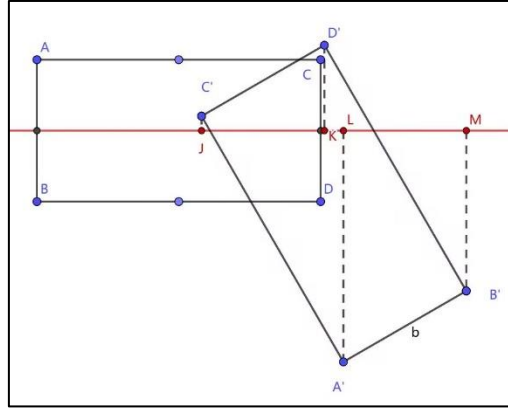


图 9：碰撞检测模型演示图

当选定了一个矩形 ABCD 的一条轴时，过另一矩形 $A'B'C'D'$ 的四个顶点分别向该轴作垂线，图中选定的是一条横轴，垂足分别是 J、K、L、M，取 $m = \max \{x_J, x_K, x_L, x_M\}$, $n = \min \{x_J, x_K, x_L, x_M\}$ ；矩形 ABCD 的四个顶点的横坐标分别为 x_A, x_B, x_C, x_D ，取 $m' = \max \{x_A, x_B, x_C, x_D\}$ ， $n' = \min \{x_A, x_B, x_C, x_D\}$ ，当 $\begin{cases} m > n' \\ n < m' \end{cases}$ 同时成立时，判断这两个矩形相碰。

5.2.4 仿真实验和算法设计

考虑到板凳龙点数众多且在运动的过程中每个点的位置都随时间变化，除第一个点外的其他点均做非匀变速曲线运动，计算的方法已很难解决面临的问题，本文采用

模拟仿真的方法寻找舞龙队不能继续盘入的时间。

为了控制时间复杂度，在盘龙的过程中无需对每个时刻众多板凳两两进行检测。逐一两两检测既增加了程序的复杂程度，又会存在大量的无效检测，因此结合前文已证明的理论（内圈比外圈容易发生碰撞），只需要对龙头和前半龙身各矩形之间两两进行检测即可。

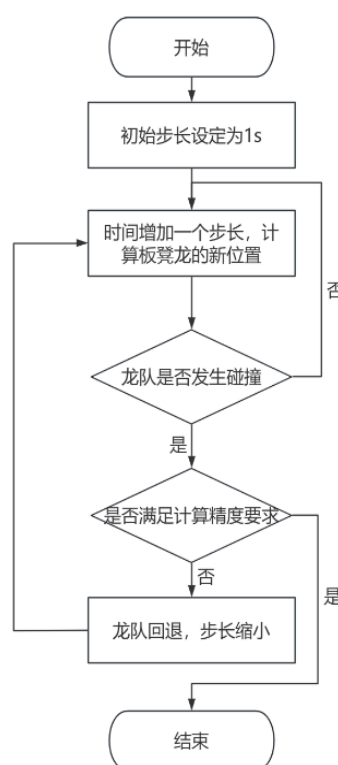


图 10 问题 2 算法流程图

STEP 1 以 $t_0 = 1$ 秒作为初始步长，从初始时刻龙头进入螺线开始计时，直至龙内第一次发生碰撞时停止，记录此时的时间 t_1 。

STEP 2 将龙头回退到 $t_1 - 10t_0$ 时刻所在位置并将步长更新为 $t'_0 = 0.1t_0$ ，时间清零后再次计时盘入直至再次发生碰撞，记录花费的时间 t_2 ，将龙头回退到 $t_2 - 10t'_0$ 所在位置并将步长更新为 $t''_0 = 0.1t'_0$ ，即 0.01 秒.....每次循环步长都更新为原步长的 0.1 倍，每次盘龙的起点都是上一次盘龙终止时刻 10 倍步长前的位置。

STEP 3 如此循环下去直至完成单位时间为 0.0001s 时的盘龙，利用问题 1 中的解法求解龙队各点的位置与速度。

根据此算法可以求得舞龙队恰好不能继续盘龙时的时间为 **412.2763s**，位置与速度如下表所示。

	横坐标 x (m)	纵坐标 y (m)	速度 (m/s)
龙头	1.211977	1.941397	1
第 1 节龙身	-1.641496	1.755397	0.991555
第 51 节龙身	1.284059	4.32568	0.976862
第 101 节龙身	-0.539221	-5.879821	0.974557
第 151 节龙身	0.965875	-6.957854	0.973614
第 201 节龙身	-7.893589	-1.227807	0.973102
龙尾（后）	0.95918	8.322364	0.972942

表 3: 指定把手位置及速度表

5.3 问题 3 的模型建立与求解

5.3.1 仿真实验和算法设计

本问的变量是螺线间距，要求解在龙队能够不发生碰撞地行进到螺线上 $\rho = 4.5m$ 处的条件下的最小螺线间距 d_{min} 。在第二问的基础上，本文使用仿真实验的方法求解本题并使用二分法寻找满足题意的最小螺线间距。

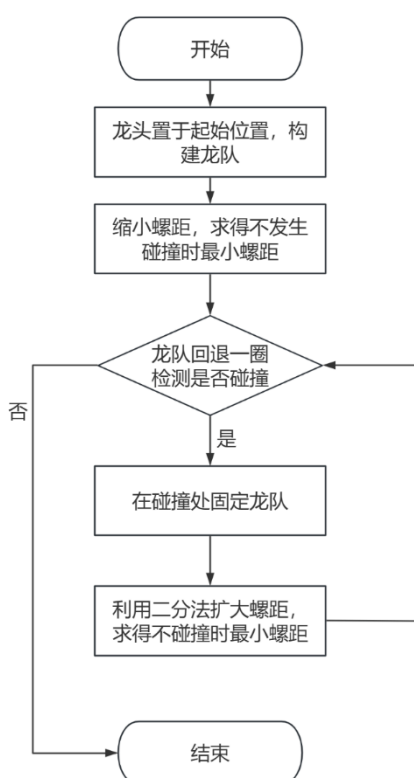


图 11: 问题 3 算法流程图

STEP 1 满足题意的螺线间距一定能够使龙队不碰撞地到达螺线上 $\rho = 4.5m$ 处，因此可以先将龙头固定在 $\rho = 4.5m$ 处求解最小螺线间距，如图所示。在寻找最小螺线间距时采用二分法更便捷地找到一个目标值(d_{solve})，但该值只满足在该静态时刻下整个龙队不碰撞，不能确保从起点到 $\rho = 4.5m$ 处的过程中龙队不发生碰撞，因此在找到该目标值后需要对整个过程进行检验。

STEP 2 检验时使龙头从圆形边缘处 ($\rho = 4.5m$) 回退至圆形边缘的螺线次外圈同点处 ($\rho = (4.5 + d_{solve})cm$) (根据 5.2.3 螺线内圈比外圈

更容易碰撞的理论，若次外圈无碰撞，则其余外圈均无碰撞)，记录回退过程中是否存

在碰撞，若不发生碰撞，则该目标值 d_{solve} 就是螺线间距的最小值；若发生碰撞，则记录碰撞位置后重新利用二分法寻找最小值。在下一次检测时，将龙头置于上次碰撞位置进行回退（保证通过检测的 $d_{solve}' > d_{solve}$ ）。

STEP3 目标值 d_{solve} 通过检测，即在该最小螺距下，整个龙队能盘入至调头区域边缘。求得该最小螺距 $D = 45.032665\text{cm}$ 。

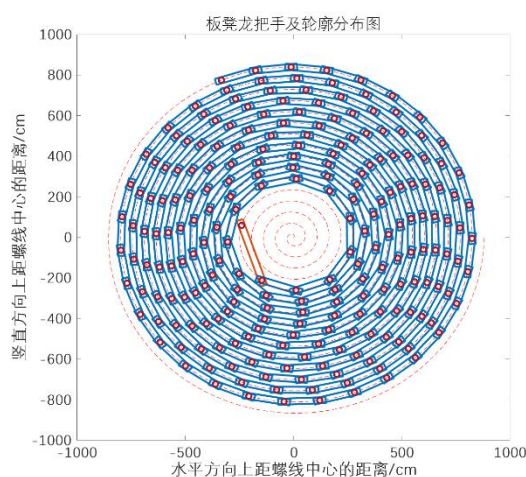


图 12 $\rho = 450\text{cm}$ 时板凳分布图

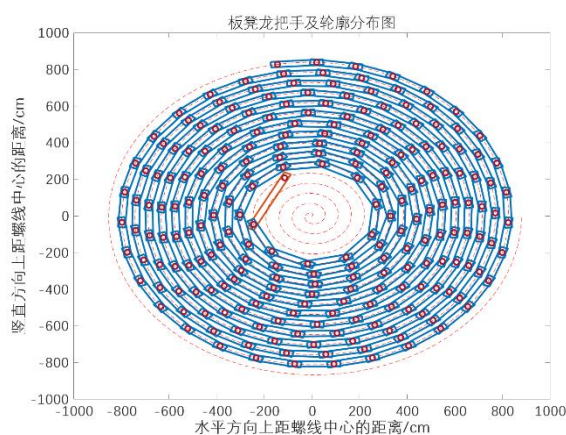


图 13 $\rho = (450 + d_{solve})\text{cm}$ 时板凳分布图

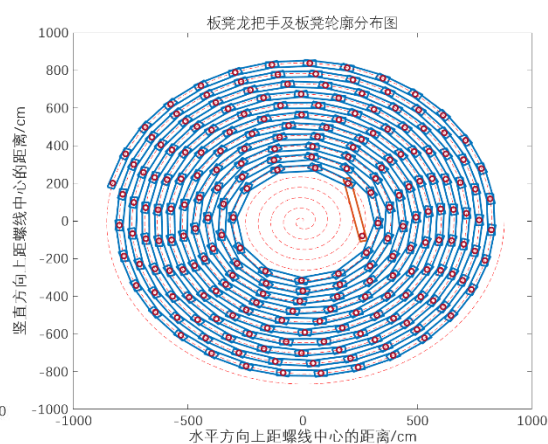


图 14 龙头在碰撞时板凳分布图

5.4 问题 4 的建模与求解

本问螺距固定，盘出螺线与盘入螺线中心对称，调头路径由两段相切圆弧组成，

前后两段圆弧半径比为 2: 1, 两段圆弧分别与盘入、盘出螺线相切。求解最小调头曲线长度, 要建立板凳龙全行进路线模型的构建, 并构建调头曲线长度函数, 同时找出约束条件。

5.4.1 板凳龙全行进路线模型的构建和几何证明

如图 15, 红色螺线为盘入螺线, 绿色螺线为盘出螺线。在盘入螺线上任取点 B, 作 B 关于 O 的中心对称点 B', 分别作 B、B' 处螺线的法线 l_{n1} 、 l_{n2} 与切线 l_{t1} 、 l_{t2} , 连接 BB' 并取点 P 使 $BP=2B'P$, 在法线 l_{n1} 上取 C_1 作过 BP 的圆, 在法线 l_{n2} 上取 C_2 作过 B'P 的圆, 此时 C_1 、 C_2 、P 三点共线且 $\odot C_1$ 、 $\odot C_2$ 相切于点 P, 证明如下:

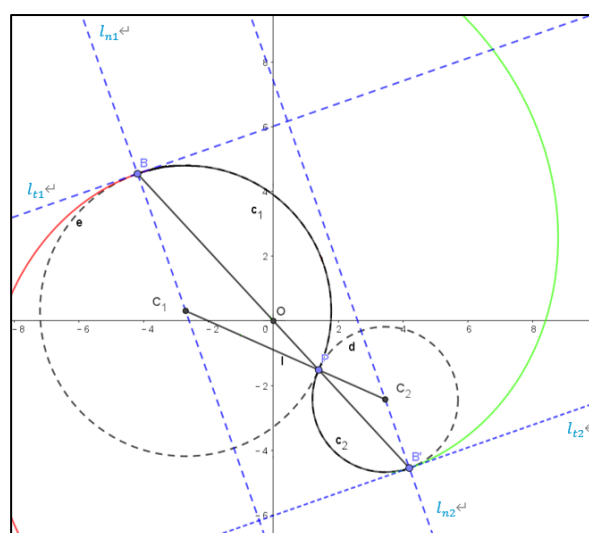


图 15: S 形曲线构建图

$$\begin{aligned}
 &\because l_{n1} // l_{n2} \\
 &\therefore \angle C_1BP = \angle C_2B'P \\
 &\because C_1B = C_1P, C_1B' = C_2P \\
 &\therefore \angle C_1BP = \angle C_1PB, \\
 &\quad \angle C_2B'P = \angle C_2PB' \\
 &\therefore \angle C_1BP = \angle C_2PB' \\
 &\therefore C_1、C_2、P \text{ 三点共线} \\
 &\quad (\text{对顶角相等}) \\
 &\therefore C_1C_2 = C_1P + C_2P
 \end{aligned}$$

5.4.2 调头曲线长度函数的构建

由几何关系, 可以计算得盘入螺线上任意 B 点所对应的 S 形曲线长度。若 B 坐标为 $(\theta, a\theta)$, 则有:

$$l = BB' = 2a\theta, \quad PB' = \frac{2a\theta}{3}$$

利用螺线方程求出直线 l_{t1} 的斜率 k, 带入点 B 坐标, 可以求得直线 l_{t1} 的解析式:

$$k = \frac{\sin \theta + \theta \cos \theta}{\cos \theta - \theta \sin \theta}$$

$$kx - y + a\theta \sin \theta - ak\theta \cos \theta = 0$$

利用点到直线距离公式，可以求得点 B' 到直线 l_{t1} 的距离，记作 d_1 ：

$$d_1 = \frac{|2a\theta \sin \theta - 2ak\theta \cos \theta|}{\sqrt{k^2 + 1}}$$

记 $\angle C_2 B' B$ 为 α ， $\angle P C_2 B'$ 为 β ， $\odot C_2$ 的半径为 r ，可得 $\cos \alpha = \frac{d_1}{l}$ ，则

$\alpha = \cos^{-1} \frac{d_1}{l}$ ，由 $\odot C_2$ 中几何关系可得：

$$\beta = \pi - 2\alpha$$

$$2PC_2 \cos \alpha = PB' \quad \text{即} \quad 2r \cos \alpha = \frac{1}{3}l \quad \text{即} \quad r = \frac{l}{6 \cos \alpha}$$

综上，我们已经得到了两段圆弧半径与其所对圆心角，可以构建 S 形曲线长度函数，记为 len_1 ：

$$len_1 = 2r\beta + r\beta = 3r\beta$$

记调头空间边界处螺线上点的极角为 θ_0 ，利用阿基米德螺线弧长公式，可以求得调头曲线中螺线部分的长度函数，记为 len_2 ：

$$len_2 = a \left[\theta_0 \sqrt{1 + \theta_0^2} + \ln \left(\theta_0 + \sqrt{1 + \theta_0^2} \right) \right] - a \left[\theta \sqrt{1 + \theta^2} + \ln \left(\theta + \sqrt{1 + \theta^2} \right) \right]$$

记整个调头曲线的长度为 len ，则 $len = len_1 + len_2$ ，由于平面几何条件的约束，本问的唯一变量是龙队进入调头空间后选择走 S 形曲线的坐标位置，也可表述为调头空间内盘入螺线与第一段圆弧线的交点（B 点）的坐标。本函数中所有变量均可用 θ 表示，即 θ 为该目标函数唯一决策变量。根据题目要求，该目标函数需要在指定 θ 范围内求到最小值。为使目标函数最小化，需要先得到 θ 的取值范围。

5.4.3 调头曲线潜在约束条件的探讨

本问可能的潜在约束条件为：B 点坐标(即 θ)改变时，调头曲线随之变化，可能导致龙队行进过程中出现**碰撞、回退等异常现象**，需要检验。

逐渐减小 θ 对该模型进行**计算机仿真检测**，龙队在行进过程中**始终不会出现碰撞的**

情况，但特定条件下部分把手速度出现负值且坐标值出现突变情况。为解决该异常，对龙头两把手运动状态进行进一步建模，延长 PC_2 交盘出螺线于D，当 $PD = l_0$ 时，达到异常状态临界条件。下图表示了临界情况下，龙头把手在盘出螺线上运动时，龙头后把手的运动状态，S代表龙头把手， $\odot S$ 代表龙头后把手的可能落点，下图展示了龙头后把手经过大圆弧、两圆切点、小圆弧的运动过程。利用 MATLAB 编程求解，得到此时的 $\theta = 15.782\text{rad}$ ，为 θ 的下界，即螺线延申进入调头区域的最长距离。

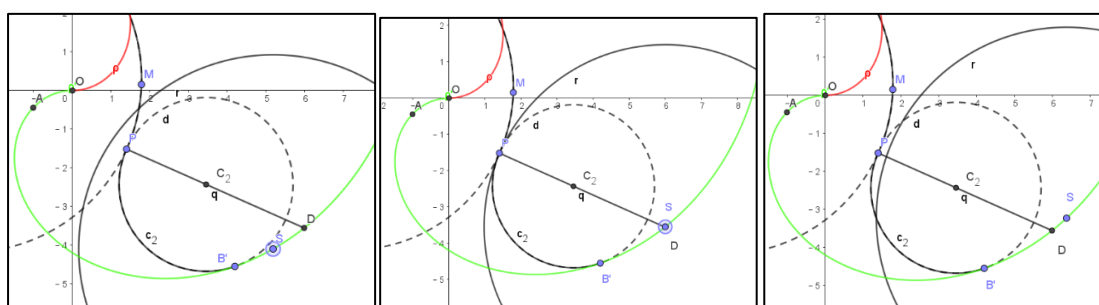


图 15: $\theta=15.782\text{rad}$ 时，龙头运动轨迹演示图

5.4.4 调头曲线长度函数最小值的求解

由于调头路径被限制在调头空间内，可以得到舞龙队一进入调头空间就立即开始沿圆弧调头所对应的起始位置: $\theta = \frac{2\pi \times 450}{170}$ ，为 θ 的上界，即螺线延申进入调头区域的最段距离。

得到自变量的取值范围后，调头曲线长度最小值求解问题边转化为了指定范围内单目标优化问题。这里得到指定范围内调头曲线长度随 θ 的变化曲线如图 17

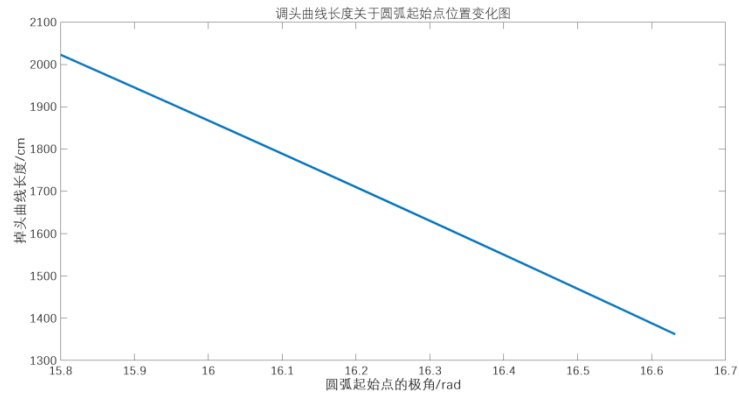


图 16：调头曲线长度关于圆弧起始点位置变化图

可以看出调头曲线长度与 θ 呈负相关，因此调头曲线长度在 θ 取到上界 $\theta = \frac{2\pi \cdot 450}{170}$

时调头曲线最短，为 **13.621m**。

得到该状态下的板凳龙

轨迹如图 17。根据该轨迹对

第一问得到的**龙头运动模型**

和**把手位置递推算法**适应性

更新，得到新的**龙头运动模**

型、把手位置递推算法，根

据新模型和算法可以求解-

200s 到 200s 间以 1s 为步长各时刻的板凳龙所有把手位置，然后对所有位置以极小时

间间隔使用**中心差分法**得到该点的速度，最后求得结果如下：

龙队位置结果表

	-100s	-50s	0s	50s	100s
龙头 x (m)	7.778034	6.608301	-2.711856	1.332696	-3.157229
龙头 y (m)	3.717164	1.898865	-3.591078	6.175324	7.548511
第 1 节龙身 x (m)	6.209273	5.366911	-0.063534	3.862265	-0.34689
第 1 节龙身 y (m)	6.108521	4.475403	-4.670888	4.840828	8.079166

第 51 节龙身 x (m)	-10.60804	-3.629945	2.459962	-1.671385	2.095033
第 51 节龙身 y (m)	2.831491	-8.9638	-7.778145	-6.076713	4.033787
第 101 节龙身 x (m)	-11.92276	10.125787	3.008493	-7.591816	-7.288774
第 101 节龙身 y (m)	-4.802378	-5.972247	10.108539	5.175487	2.063875
第 151 节龙身 x (m)	-14.35103	12.974784	-7.002789	-4.605165	9.462513
第 151 节龙身 y (m)	-1.980993	-3.810357	10.337482	-10.38699	-3.540357
第 201 节龙身 x (m)	-11.95294	10.522509	-6.872842	0.336952	8.524374
第 201 节龙身 y (m)	10.566998	-10.80743	12.382609	-13.17761	8.606933
龙尾 (后) x (m)	-1.011059	0.189809	-1.933627	5.859094	-10.980157
龙尾 (后) y (m)	-16.52757	15.720588	-14.713128	12.612894	-6.770006

表 4：龙队位置结果表

龙队速度结果表

	-100s	-50s	0s	50s	100s
龙头 (m/s)	1	1	1	1	1
第 1 节龙身 (m/s)	0.999904	0.999762	0.998686	1.000363	1.000124
第 51 节龙身 (m/s)	0.999346	0.998642	0.995134	0.949935	1.003966
第 101 节龙身 (m/s)	0.999091	0.998248	0.994447	0.948482	1.096263
第 151 节龙身 (m/s)	0.998944	0.998047	0.994156	0.948038	1.095306
第 201 节龙身 (m/s)	0.998849	0.997925	0.993994	0.947823	1.094933
龙尾 (后) (m/s)	0.998817	0.997885	0.993944	0.94776	1.094833

表 5：龙队速度结果表

5.5 问题 5 的建模与求解

本问要求在第四问建立的板凳龙全行进路线模型的基础上求解龙头把手速度的最大值，属于最优化问题。考虑到本问要求模拟全过程速度，计算量较大，同时计算机仿真计算把手速度的精度与时间步长有较大关联，为保证算法的正确性与可行性，建立以下模型并求解。

5.5.1 速度比值计算模型的证明与建立：

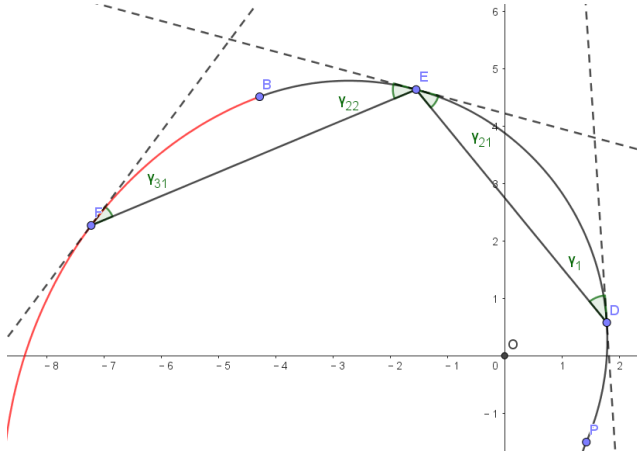


图 18：速度比值计算模型证明图

图例说明：

如图 19, γ_1 、 γ_{21} 、 γ_{22} 、 γ_{31} 分别为 D、E、F 三点处切线与 ED、EF 的夹角。

由第二问提出的速度传递算法可以列出以下方程：

$$\begin{cases} v_D \cos \gamma_1 = v_E \cos \gamma_{21} \\ v_E \cos \gamma_{22} = v_F \cos \gamma_{31} \end{cases}$$

由此可以求得：

$$\begin{cases} v_E = \frac{v_D \cos \gamma_1}{\cos \gamma_{21}} \\ v_F = \frac{v_D \cos \gamma_1 \cos \gamma_{22}}{\cos \gamma_{21} \cos \gamma_{31}} \end{cases}$$

以此类推，可以求得任意一点 N 的速度为 $v_N = v_D \frac{\cos \gamma_1 \cos \gamma_{22}}{\cos \gamma_{21} \cos \gamma_{31}} \dots \frac{\cos \gamma_{(n-1)2}}{\cos \gamma_{n1}}$ ，即当板

凳龙结构特征相同（即各把手坐标分别相等）时，**龙头速度与任一把手速度呈定比。**

由此可以构建速度比值计算模型。

速度比值计算模型：对任一龙队结构，利用上述几何关系求得该结构下龙队任意把手速度与龙头速度的比值为 $\frac{\cos \gamma_1 \cos \gamma_{22}}{\cos \gamma_{21} \cos \gamma_{31}} \dots \frac{\cos \gamma_{(n-1)2}}{\cos \gamma_{n1}}$ ，其中各 γ 角表示各把手处切线与板凳的夹角。

5.5.2 利用速度比值计算模型求解龙头最大速度：

龙身各把手速度受制于龙头速度与龙队结构，利用**速度比值计算模型**，可以将本问简化为求解各龙队结构下**速度比值的最大值**，从而**略去龙头速度影响**，极大降低计算复杂度。

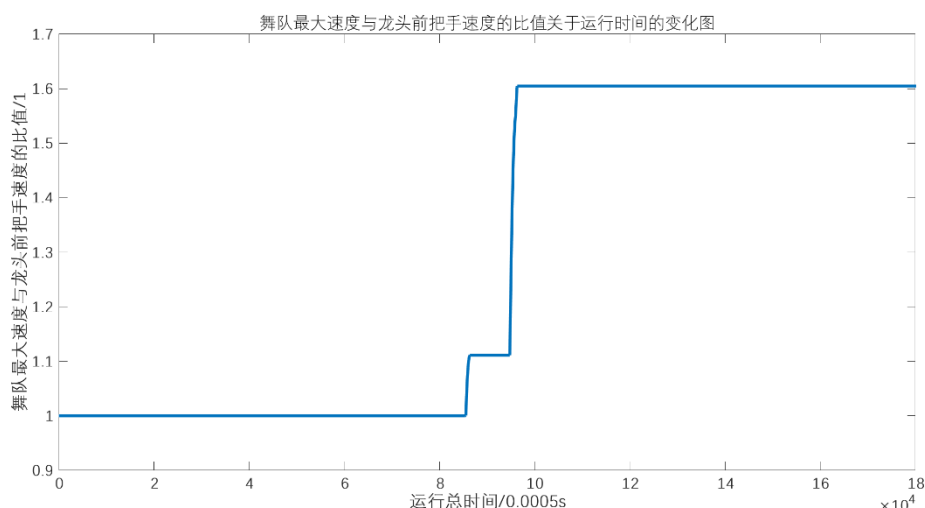


图 19：龙队最大速度与龙头前把手速度的比值关系关于时间的变化图

为提高速度比值计算精度并简化计算，采用微分法计算各结构下各把手速度，从而得到速度比值。令龙头速度为 1m/s，时间步长为 0.0005s（即每 0.0005s 计算一次龙队结构与各把手速度），采用贪心算法的思想求取每一个步长下的最大速度比值，再与全局最大速度比值比较后更新，求得全局最大速度比值为 **1.60479**，为保证全部把手速度不超过 2m/s，龙头把手的最大速度 $v=1.2463\text{m/s}$ 。

六、模型的评价

6.1 模型的优点

1. 本文的模型大多采用了二分法的计算方法，精度可调。在对于精确度和误差没有过高要求时可以降低精确度以换取程序更低的时间复杂度；在对于精度和误差有较高要求时，又可上调精度以满足要求，适用性强。
2. 本文的模型给出了求解一系列串连的点在曲线上运动的通用思路，适用性强。无论曲线的几何形状如何变化，位置递推算法、速度传递算法以及差分法求解速度都是解决该类距离为定长的点做曲线运动问题的通用解法。
3. 本文建立的调头曲线几何模型给出了严谨的数学证明，排除了绘图和求解时的不确定性以及误差，支撑了模型的正确性，增强了模型的说服力。

6.2 模型的缺点

- 1.在使用位置递推算法和速度传递算法时计算量大，逐点递推的计算方式较为复杂。
- 2.在个别计算和证明中，本模型将外圈螺线近似等价于圆进行处理，存在误差和不严谨之处。

6.3 模型的推广

首先，模型给出了曲线上各个关联点位置和速度随时间变化的关系，可应用于各点阵类的节目演出，例如无人机夜空灯光秀，军队方阵表演，社团组织多人节目演出等。其次，模型可应用于机械臂和机器人运动的制造与研发。板凳龙相邻两节板凳在运动过程中高度还原了机械臂的运动状态，即相邻的两实体结构通过贯穿二者的轴相连，只能通过改变角度到达目标位置。最后，本模型还可应用于航空航天类飞行器的设计与研究，在宇宙中航天飞行器上各关联点也是通过牵引在设计好的曲线上做非匀变速曲线运动，与板凳龙的运动过程高度相似。

七、参考文献

- [1]. 宫明法,高博,牛秋丽.基于特征点的快速碰撞检测算法 [J]. 图学学报. 2008,第006 期
- [2]. 高博.基于特征点的碰撞检测算法的研究 [A]. 2008
- [3]. 于文勤.超静定梁的一种计算方法——中心差分法. 青岛理工大学学报.1989. 第 2 期 78-86

附录1

题目解决所用代码

<i>circle_solve.m</i>	函数文件，输入 θ 输出两段圆弧的信息
<i>five_solve0.m</i>	第五问解题主程序
<i>four_point.m</i>	函数文件，输入把手极距输出板凳四角的位置
<i>four_point_plus.m</i>	函数文件，输入把手笛卡尔坐标输出板凳四角位置
<i>four_solve0.m</i>	第四问解题主程序
<i>hand_move.m</i>	函数文件，不调头时的龙头运动模型
<i>hand_move_plus_pro.m</i>	函数文件，存在调头时的龙头运动模型
<i>is_bang.m</i>	函数文件，碰撞检测模型
<i>my_rad.m</i>	函数文件，夹角计算函数
<i>next_point.m</i>	函数文件，存在调头时的把手递推模型
<i>next_v.m</i>	函数文件，不调头时速度递推模型
<i>o_to_d.m</i>	函数文件，输入 θ 输出最大容纳把手间距
<i>o_to_l.m</i>	函数文件，输入 θ 输出调头曲线长度
<i>one_solve0.m</i>	第一问解题主程序
<i>three_solve0.m</i>	第三问解题主程序
<i>two_solve0.m</i>	第二问解题主程序

附录2

circle_solve.m 内容

```
function out = circle_solve(o_start)

f1 = @(x) 170.*x./(2*pi);

x_in = f1(o_start)*cos(o_start);
y_in = f1(o_start)*sin(o_start);
x_out = -x_in;
y_out = -y_in;

l = (170/pi)*o_start;

k0 = (sin(o_start)+o_start*cos(o_start))/(cos(o_start)-o_start*sin(o_start));
k_t = -1/k0;
```

```

d2 = abs(-2*(y_in)+2*k0*x_in)/(k0^2+1)^0.5;

cos_a = d2/(1);

r_s = (1/2)/(3*cos_a);

r_l = 2*r_s;

b_l = y_in-k_t*x_in;

b_s = y_out-k_t*x_out;

c_l_x_1 = x_in-r_l/(k_t^2+1)^0.5;
c_l_x_2 = x_in+r_l/(k_t^2+1)^0.5;
c_l_y_1 = k_t*c_l_x_1+b_l;
c_l_y_2 = k_t*c_l_x_2+b_l;
if (c_l_y_2^2+c_l_x_2^2)>(c_l_y_1^2+c_l_x_1^2)
c_l_x = c_l_x_1;
c_l_y = c_l_y_1;
else
c_l_x = c_l_x_2;
c_l_y = c_l_y_2;
end

c_s_x_1 = x_out-r_s/(k_t^2+1)^0.5;
c_s_x_2 = x_out+r_s/(k_t^2+1)^0.5;
c_s_y_1 = k_t*c_s_x_1+b_s;
c_s_y_2 = k_t*c_s_x_2+b_s;
if (c_s_y_2^2+c_s_x_2^2)>(c_s_y_1^2+c_s_x_1^2)
c_s_x = c_s_x_1;

```

```

c_s_y = c_s_y_1;

else

c_s_x = c_s_x_2;

c_s_y = c_s_y_2;

end

x_i = (2*c_s_x+c_l_x)/3;

y_i = (2*c_s_y+c_l_y)/3;

o_c = pi-2*acos(cos_a);


out = [c_l_x,c_l_y,r_l
c_s_x,c_s_y,r_s
x_i,y_i,o_c];

end

```

附录3

five_solve0.m 内容

```

clear

o_start = 450*2*pi/170;

r0 = 170/(2*pi)*o_start+170;

v0 = 100;

move_xy = zeros(224,3);

move_xys = zeros(224,2,2);

T = 0.0005;


out = circle_solve(o_start);

c_l_x = out(1,1);

```

```

c_l_y = out(1,2);

r_l = out(1,3);

c_s_x = out(2,1);

c_s_y = out(2,2);

r_s = out(2,3);

x_i = out(3,1);

y_i = out(3,2);

o_c = out(3,3);


for t = 0:1

out_point = hand_move_plus_pro(out,r0,o_start,t*T,v0);

move_xy(1,:) = out_point;

out_point = next_point(out_point(1),out_point(2),out_point(3),out,286,o_start);

move_xy(2,:) = out_point;

for i = 1:222

out_point = next_point(out_point(1),out_point(2),out_point(3),out,165,o_start);

move_xy(i+2,:) = out_point;

end

move_xys(:,1:2,t+1) = move_xy(:,1:2);

end


vs = (((move_xys(:,1,1)-move_xys(:,1,2)).^2+(move_xys(:,2,1)-
move_xys(:,2,2)).^2).^0.5)/T;

v_max = max(vs);

v_maxs = v_max;

for t = 2*T:T:90

out_point = hand_move_plus_pro(out,r0,o_start,t,v0);

```

```

move_xy(1,:) = out_point;

out_point = next_point(out_point(1),out_point(2),out_point(3),out,286,o_start);

move_xy(2,:) = out_point;

for i = 1:222

out_point = next_point(out_point(1),out_point(2),out_point(3),out,165,o_start);

move_xy(i+2,:) = out_point;

end

move_xys(:,1:2,1) = move_xys(:,1:2,2);

move_xys(:,1:2,2) = move_xy(:,1:2);

vs = (((move_xys(:,1,1)-move_xys(:,1,2)).^2+(move_xys(:,2,1)-
move_xys(:,2,2)).^2).^0.5)/T;

v_max_t = max(vs);

if v_max_t>v_max

v_max = v_max_t;

end

v_maxs = [v_maxs,v_max];

end

plot(v_maxs/100);

disp(v_max)

```

附录4

four_point.m 内容

```

function out = four_point(in)

out = zeros(4,2);

r1 = in(1);

r2 = in(2);

o1 = r1*2*pi/55;

```

```

o2 = r2*2*pi/55;

x1 = r1*cos(o1);
y1 = r1*sin(o1);
x2 = r2*cos(o2);
y2 = r2*sin(o2);

if x1==x2

out(1,1) = x1 - 15;
out(2,1) = x1 - 15;
out(3,1) = x1 + 15;
out(4,1) = x1 + 15;
yn = min(y1,y2);
yx = max(y1,y2);
out(1,2) = yx + 27.5;
out(2,2) = yx - 27.5;
out(3,2) = yn - 27.5;
out(4,2) = yn + 27.5;

end

if y1==y2

out(1,2) = y1 + 27.5;
out(2,2) = y1 - 27.5;
out(3,2) = y1 - 27.5;
out(4,2) = y1 + 27.5;
xn = min(x1,x2);
xx = max(x1,x2);
out(1,1) = xn - 15;

```

```

out(2,1) = xn - 15;

out(3,1) = xx + 15;

out(4,1) = xx + 15;

end

k = (y2 - y1)/(x2 - x1);

b = y2 - k*x2;

bn = b-15*(k^2+1)^0.5;

bx = b+15*(k^2+1)^0.5;

xn = min(x1,x2);

xx = max(x1,x2);

x0 = xn - 27.5/(k^2+1)^0.5;

x3 = xx + 27.5/(k^2+1)^0.5;

bt0 = k*x0 + b - (-1/k)*x0;

bt1 = k*x3 + b - (-1/k)*x3;

out(1,1) = (bt0-bx)/(k+1/k);

out(1,2) = k*out(1,1)+bx;

out(2,1) = (bt0-bn)/(k+1/k);

out(2,2) = k*out(2,1)+bn;

out(3,1) = (bt1-bn)/(k+1/k);

out(3,2) = k*out(3,1)+bn;

out(4,1) = (bt1-bx)/(k+1/k);

out(4,2) = k*out(4,1)+bx;

```

end

附录5

four_point_plus.m 内容

```
function out = four_point_plus(in,d)

out = zeros(4,2);

r1 = in(1);
r2 = in(2);

o1 = r1*2*pi/d;
o2 = r2*2*pi/d;

x1 = r1*cos(o1);
y1 = r1*sin(o1);
x2 = r2*cos(o2);
y2 = r2*sin(o2);

if x1==x2

out(1,1) = x1 - 15;
out(2,1) = x1 - 15;
out(3,1) = x1 + 15;
out(4,1) = x1 + 15;

yn = min(y1,y2);
yx = max(y1,y2);

out(1,2) = yx + 27.5;
out(2,2) = yx - 27.5;
out(3,2) = yn - 27.5;
out(4,2) = yn + 27.5;

end
```

```

if y1==y2

out(1,2) = y1 + 27.5;

out(2,2) = y1 - 27.5;

out(3,2) = y1 - 27.5;

out(4,2) = y1 + 27.5;

xn = min(x1,x2);

xx = max(x1,x2);

out(1,1) = xn - 15;

out(2,1) = xn - 15;

out(3,1) = xx + 15;

out(4,1) = xx + 15;

end


k = (y2 - y1)/(x2 - x1);

b = y2 -k*x2;

bn = b-15*(k^2+1)^0.5;

bx = b+15*(k^2+1)^0.5;

xn = min(x1,x2);

xx = max(x1,x2);

x0 = xn - 27.5/(k^2+1)^0.5;

x3 = xx + 27.5/(k^2+1)^0.5;

bt0 = k*x0 + b - (-1/k)*x0;

bt1 = k*x3 + b - (-1/k)*x3;

out(1,1) = (bt0-bx)/(k+1/k);

out(1,2) = k*out(1,1)+bx;

```

```

out(2,1) = (bt0-bn)/(k+1/k);

out(2,2) = k*out(2,1)+bn;


out(3,1) = (bt1-bn)/(k+1/k);

out(3,2) = k*out(3,1)+bn;


out(4,1) = (bt1-bx)/(k+1/k);

out(4,2) = k*out(4,1)+bx;

end

```

附录6

four_solve0.m 内容

```

clear

f1 = @(x) 170.*x./(2*pi);

o_limit = 2*pi*450/170;

v = 100;

TT = 200;

T = 0.00001;


len = 10000;

o = linspace(0,o_limit,len);

ls = zeros(1,len);

ds = zeros(1,len);

for i = 1:len

ls(i) = o_to_l(o(i));

ds(i) = o_to_d(o(i)+0.0001);

end

```

```

plot(o,ls);

figure

plot(o,ds);


o_start = o_limit;


f1 = @(x)(170/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5));

l1 = v*(TT/2+1)+f1(o_start);

f = @(x)(170/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5))-l1;

o0 = fzero(f,o_start);

r0 = f1(o0);


index_x = 1:2:447;

index_y = 2:2:448;

ii = [-1 1];


move_xy = zeros(224,3);

move_xys = zeros(224,2,2);

xys = zeros(448,TT+1);

vs = zeros(224,TT+1);


out = circle_solve(o_start);


for t = 1:TT+1

out_point = hand_move_plus_pro(out,r0,o_start,t,v);

move_xy(1,:) = out_point;

```

```

out_point = next_point(out_point(1),out_point(2),out_point(3),out,286,o_start);
move_xy(2,:) = out_point;

for i = 1:222
out_point = next_point(out_point(1),out_point(2),out_point(3),out,165,o_start);
move_xy(i+2,:) = out_point;
end

xys(index_x,t) = move_xy(:,1);
xys(index_y,t) = move_xy(:,2);


for iii = 1:2
out_point = hand_move_plus_pro(out,r0,o_start,t+ii(iii)*T,v);
move_xy(1,:) = out_point;
out_point = next_point(out_point(1),out_point(2),out_point(3),out,286,o_start);
move_xy(2,:) = out_point;

for i = 1:222
out_point = next_point(out_point(1),out_point(2),out_point(3),out,165,o_start);
move_xy(i+2,:) = out_point;
end

move_xys(:,1:2,iii) = move_xy(:,1:2);

end

vs(:,t) = (((move_xys(:,1,1)-move_xys(:,1,2)).^2+(move_xys(:,2,1)-
move_xys(:,2,2)).^2).^0.5)/(2*T);

end

xys = roundn(xys/100,-6);

vs = roundn(vs/100,-6);

need1 = [-100, -50 0 50 100]+101;

```

```

need2 = zeros(1,7);

need2_t1 = 1:2:13;

need2_t2 = 2:2:14;

need2_t = [0 1 51 101 151 201 223];

need2(need2_t1) = 2*need2_t+1;

need2(need2_t2) = 2*need2_t+2;

xxyy = xys(need2,need1);


vvv = vs(need2_t+1,need1);

```

附录7

hand_move.m 内容

```

function out = hand_move(r0,t)

v = 100;

x0 = 2*pi*r0/55;

l0 = (55/(4*pi))*(x0*(1+x0^2)^0.5+log(x0+(1+x0^2)^0.5));

f = @(x)(55/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5))+v*t-l0;

out = 55*fzero(f,x0)/(2*pi);

end

```

附录8

hand_move_plus_pro.m 内容

```

function out = hand_move_plus_pro(in,r_start,o_start,t,v)

out = zeros(1,3);

f1 = @(x) 170.*x./(2*pi);

f2 = @(x) -170.*(x)./(2*pi);

f1 = @(x)(170/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5));

l1 = f1(r_start*2*pi/170)-f1(o_start);

c_l_x = in(1,1);

```

```

c_l_y = in(1,2);

r_l = in(1,3);

c_s_x = in(2,1);

c_s_y = in(2,2);

r_s = in(2,3);

x_i = in(3,1);

y_i = in(3,2);

o_c = in(3,3);

o_l = my_rad(x_i-c_l_x,y_i-c_l_y);

o_s = my_rad(x_i-c_s_x,y_i-c_s_y);

if v*t< ll

x0 = 2*pi*r_start/170;

l0 = (170/(4*pi))*(x0*(1+x0^2)^0.5+log(x0+(1+x0^2)^0.5));

f = @(x)(170/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5))+v*t-l0;

o = fzero(f,x0);

out(1) = f1(o)*cos(o);

out(2) = f1(o)*sin(o);

out(3) = 1;

elseif v*t < ll+o_c*r_l

r_o_l = (v*t- ll)/r_l;

out(1) = r_l*cos(o_l+o_c-r_o_l)+c_l_x;

out(2) = r_l*sin(o_l+o_c-r_o_l)+c_l_y;

out(3) = 2;

elseif v*t<ll+o_c*r_l+o_c*r_s

r_o_s = (v*t-r_l*o_c- ll)/r_s;

out(1) = r_s*cos(o_s+r_o_s)+c_s_x;

```

```

out(2) = r_s*sin(o_s+r_o_s)+c_s_y;

out(3) = 3;

else

f = @(x)(170/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5))-(v*t-11-o_c*(r_l+r_s))-
f1(o_start);

o = fzero(f,0);

out(1) = f2(o)*cos(o);

out(2) = f2(o)*sin(o);

out(3) = 4;

end

end

```

附录9

is_bang.m 内容

```

function out = is_bang(in1,in2)

%一号矩形一号轴

k = (in1(1,2)-in1(2,2))/(in1(1,1)-in1(2,1));

b = in1(1,2)-k*in1(1,1);

xs = zeros(1,4);

for i = 1:4

xs(i) = ((1/k)*in2(i,1)+in2(i,2)-b)/(k+1/k);

end

if ~(min(xs)<max(in1(1,1),in1(2,1))&&max(xs)>min(in1(1,1),in1(2,1)))

out = 0;

return;

end

```

```

% 一号矩形二号轴

k = (in1(2,2)-in1(3,2))/(in1(2,1)-in1(3,1));

b = in1(2,2)-k*in1(2,1);

for i = 1:4

xs(i) = ((1/k)*in2(i,1)+in2(i,2)-b)/(k+1/k);

end

if ~(min(xs)<max(in1(3,1),in1(2,1))&&max(xs)>min(in1(3,1),in1(2,1)))

out = 0;

return;

end

%二号矩形一号轴

k = (in2(1,2)-in2(2,2))/(in2(1,1)-in2(2,1));

b = in2(1,2)-k*in2(1,1);

for i = 1:4

xs(i) = ((1/k)*in1(i,1)+in1(i,2)-b)/(k+1/k);

end

if ~(min(xs)<max(in2(1,1),in2(2,1))&&max(xs)>min(in2(1,1),in2(2,1)))

out = 0;

return;

end

%二号矩形二号轴

k = (in2(3,2)-in2(2,2))/(in2(3,1)-in2(2,1));

b = in2(2,2)-k*in2(2,1);

```

```

for i = 1:4

xs(i) = ((1/k)*in1(i,1)+in1(i,2)-b)/(k+1/k);

end

if ~(min(xs)<max(in2(3,1),in2(2,1))&&max(xs)>min(in2(3,1),in2(2,1)))

out = 0;

return;

end

out = 1;

return

end

```

附录 10

my_rad.m 内容

```

function out = my_rad(x,y)

if x==0

if y>0

out = pi/2;

else

out = -pi/2;

end

elseif x<0&&y>0

out = pi+atan(y/x);

elseif x<0&&y<0

out = atan(y/x)-pi;

```

```
else
```

```
out = atan(y/x);
```

```
end
```

```
end
```

附录 11

next_point.m 内容

```
function out = next_point(x1,y1,in_select,in,lll,o_start)
```

```
out = zeros(1,3);
```

```
f1 = @(x) 170.*x./(2*pi);
```

```
f1 = @(x)(170/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5));
```

```
c_l_x = in(1,1);
```

```
c_l_y = in(1,2);
```

```
r_l = in(1,3);
```

```
c_s_x = in(2,1);
```

```
c_s_y = in(2,2);
```

```
r_s = in(2,3);
```

```
x_i = in(3,1);
```

```
y_i = in(3,2);
```

```
o_c = in(3,3);
```

```
if in_select==1
```

```
r0 = (x1^2+y1^2)^0.5;
```

```
f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/170-2*pi*r0/170)-lll^2;
```

```
r_limit = [r0 r0+85];
```

```
r1 = fzero(f,r_limit);
```

```

out(1) = r1*cos(r1*2*pi/170);

out(2) = r1*sin(r1*2*pi/170);

out(3) = 1;

elseif in_select==2

x_temp = f1(o_start)*cos(o_start);
y_temp = f1(o_start)*sin(o_start);
l_temp = (x1-x_temp)^2+(y1-y_temp)^2;

if l11^2>l_temp

r0 = (x1^2+y1^2)^0.5;

f = @(r) (r*cos(r*2*pi/170)-x1)^2+(r*sin(r*2*pi/170)-y1)^2-l11^2;

r_limit = [f1(o_start) f1(o_start)+85];

r1 = fzero(f,r_limit);

out(1) = r1*cos(r1*2*pi/170);

out(2) = r1*sin(r1*2*pi/170);

out(3) = 1;

else

o_l_now = my_rad(x1-c_l_x,y1-c_l_y);

r_o_l = 2*asin((l11/2)/r_l);

out(1) = r_l*cos(o_l_now + r_o_l)+c_l_x;

out(2) = r_l*sin(o_l_now + r_o_l)+c_l_y;

out(3) = 2;

end

elseif in_select==3

l_temp = (x1-x_i)^2+(y1-y_i)^2;

if l11^2>l_temp

o_l_now = my_rad(x1-c_l_x,y1-c_l_y);

```

```

l1 = ((x1-c_l_x)^2+(y1-c_l_y)^2)^0.5;

o_temp = acos((l1^2+r_l^2-l1l^2)/(2*l1*r_l));

out(1) = r_l*cos(o_l_now + o_temp)+c_l_x;

out(2) = r_l*sin(o_l_now + o_temp)+c_l_y;

out(3) = 2;

else

o_s_now = my_rad(x1-c_s_x,y1-c_s_y);

r_o_s = 2*asin((l1l/2)/r_s);

out(1) = r_s*cos(o_s_now - r_o_s)+c_s_x;

out(2) = r_s*sin(o_s_now - r_o_s)+c_s_y;

out(3) = 3;

end

elseif in_select==4

f = @(r) (-r*cos(r*2*pi/170)+f1(o_start)*cos(o_start))^2+(-
r*sin(r*2*pi/170)+f1(o_start)*sin(o_start))^2-l1l^2;

r_limit = [f1(o_start) f1(o_start)+85];

r1 = fzero(f,r_limit);

r_temp = (x1^2+y1^2)^0.5;

if r_temp<r1

o_s_now = my_rad(x1-c_s_x,y1-c_s_y);

l1 = ((x1-c_s_x)^2+(y1-c_s_y)^2)^0.5;

cos_a = (l1^2+r_s^2-l1l^2)/(2*l1*r_s);

if cos_a<-1||cos_a>1

o_l_now = my_rad(x1-c_l_x,y1-c_l_y);

l1 = ((x1-c_l_x)^2+(y1-c_l_y)^2)^0.5;

o_temp = acos((l1^2+r_l^2-l1l^2)/(2*l1*r_l));

out(1) = r_l*cos(o_l_now + o_temp)+c_l_x;

```

```

out(2) = r_l*sin(o_l_now + o_temp)+c_l_y;

out(3) = 2;

else

o_temp = acos(cos_a);

l2 = ((x1+f1(o_start)*cos(o_start))^2+(y1+f1(o_start)*sin(o_start))^2)^0.5;

o_ss = acos((l1^2+r_s^2-l2^2)/(2*r_s*l1));

if o_temp > o_c+o_ss

o_l_now = my_rad(x1-c_l_x,y1-c_l_y);

l1 = ((x1-c_l_x)^2+(y1-c_l_y)^2)^0.5;

o_temp = acos((l1^2+r_l^2-l1l^2)/(2*l1*r_l));

out(1) = r_l*cos(o_l_now + o_temp)+c_l_x;

out(2) = r_l*sin(o_l_now + o_temp)+c_l_y;

out(3) = 2;

else

out(1) = r_s*cos(o_s_now - o_temp)+c_s_x;

out(2) = r_s*sin(o_s_now - o_temp)+c_s_y;

out(3) = 3;

end

end

else

r0 = (x1^2+y1^2)^0.5;

f = @(r) (-r*cos(r*2*pi/(170))-x1)^2+ (-r*sin(r*2*pi/(170))-y1)^2-l1l^2;

r_limit = [r0 r0-85];

r1 = fzero(f,r_limit);

out(1) = -r1*cos(r1*2*pi/(170));

out(2) = -r1*sin(r1*2*pi/(170));

```

```
out(3) = 4;
```

```
end
```

```
end
```

附录 12

next_v.m 内容

```
function out = next_v(in,v)
```

```
o1 = in(1)*2*pi/55;
```

```
o2 = in(2)*2*pi/55;
```

```
x1 = in(1)*cos(o1);
```

```
y1 = in(1)*sin(o1);
```

```
x2 = in(2)*cos(o2);
```

```
y2 = in(2)*sin(o2);
```

```
k1 = (sin(o1)+o1*cos(o1))/(cos(o1)-o1*sin(o1));
```

```
k2 = (sin(o2)+o2*cos(o2))/(cos(o2)-o2*sin(o2));
```

```
k = (y2-y1)/(x2-x1);
```

```
c1 = abs((k1*k+1)/(((k^2+1)^0.5)*((k1^2+1)^0.5)));
```

```
c2 = abs((k2*k+1)/(((k^2+1)^0.5)*((k2^2+1)^0.5)));
```

```
out = v*c1/c2;
```

```
end
```

附录 13

o_to_d.m 内容

```

function out = o_to_d(o_start)

f1 = @(x) 170.*x./(2*pi);
f2 = @(x) -170.*x./(2*pi);

x_in = f1(o_start)*cos(o_start);
y_in = f1(o_start)*sin(o_start);
x_out = -x_in;
y_out = -y_in;

l = (170/pi)*o_start;

k0 = (sin(o_start)+o_start*cos(o_start))/(cos(o_start)-o_start*sin(o_start));
k_t = -1/k0;

d2 = abs(-2*(y_in)+2*k0*x_in)/(k0^2+1)^0.5;
cos_a = d2/(l);
r_s = (l/2)/(3*cos_a);
r_l = 2*r_s;

b_l = y_in-k_t*x_in;
b_s = y_out-k_t*x_out;

c_l_x_1 = x_in-r_l/(k_t^2+1)^0.5;
c_l_x_2 = x_in+r_l/(k_t^2+1)^0.5;
c_l_y_1 = k_t*c_l_x_1+b_l;
c_l_y_2 = k_t*c_l_x_2+b_l;

if (c_l_y_2^2+c_l_x_2^2)>(c_l_y_1^2+c_l_x_1^2)

```

```

c_l_x = c_l_x_1;
c_l_y = c_l_y_1;
else
c_l_x = c_l_x_2;
c_l_y = c_l_y_2;
end

c_s_x_1 = x_out-r_s/(k_t^2+1)^0.5;
c_s_x_2 = x_out+r_s/(k_t^2+1)^0.5;
c_s_y_1 = k_t*c_s_x_1+b_s;
c_s_y_2 = k_t*c_s_x_2+b_s;
if (c_s_y_2^2+c_s_x_2^2)>(c_s_y_1^2+c_s_x_1^2)
c_s_x = c_s_x_1;
c_s_y = c_s_y_1;
else
c_s_x = c_s_x_2;
c_s_y = c_s_y_2;
end

x_i = (2*c_s_x+c_l_x)/3;
y_i = (2*c_s_y+c_l_y)/3;

k = (y_i-c_s_y)/(x_i-c_s_x);
b = y_i-k*x_i;

ffff = @(x) (-170)/(2*pi)*x*sin(x)+170*k/(2*pi)*x*cos(x)-b;
xx = fzero(ffff,[o_start,o_start+pi]);
xxx = f2(xx)*cos(xx);

```

```

yyy = f2(xx)*sin(xx);

out = ((x_i-xxx)^2+(y_i-yyy)^2)^0.5;

end

```

附录 14

o_to_l.m 内容

```

function lr = o_to_l(o_start)

f1 = @(x) 170.*x./(2*pi);

f1 = @(x)(170/(4*pi))*(x*(1+x^2)^0.5+log(x+(1+x^2)^0.5));

x_in = f1(o_start).*cos(o_start);

y_in = f1(o_start).*sin(o_start);


l = (170/pi)*o_start;


k0 = (sin(o_start)+o_start.*cos(o_start))./(cos(o_start)-o_start.*sin(o_start));

d2 = abs(-2*(y_in)+2*k0.*x_in)/(k0.^2+1).^0.5;

cos_a = d2./(1);

r_s = (l/2)./(3*cos_a);


o_c = pi-2.*acos(cos_a);


lr = 3*o_c.*r_s;

lr = lr+2*(f1(450*2*pi/170)-f1(o_start));

end

```

附录 15

one_solve0.m 内容

```

clear

R0 = 16*55; %起点

T = 300; %运动总时间


fx = @(r) r.*cos(2*pi.*r/55);
fy = @(r) r.*sin(2*pi.*r/55);
index_x = 1:2:447;
index_y = 2:2:448;


rs = zeros(T+1,224);
xys = zeros(448,T+1);
vs = zeros(224,T+1);


for t = 0:T

r0 = hand_move(R0,t);

rs(t+1,1) = r0;

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/55-2*pi*r0/55)-81769;

r_limit = [r0 r0+27.5];

r1 = fzero(f,r_limit);

rs(t+1,2) = r1;

for i = 3:224

r0 = r1;

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/55-2*pi*r0/55)-27225;

r_limit = [r0 r0+27.5];

r1 = fzero(f,r_limit);

rs(t+1,i) = r1;

```

```

end

end

for t = 0:T
xys(index_x,t+1) = roundn(fx(rs(t+1,:))'/100,-6);
xys(index_y,t+1) = roundn(fy(rs(t+1,:))'/100,-6);
end

for t = 0:T
vs(1,t+1) = 1;
for i = 2:224
vs(i,t+1) = roundn(next_v(rs(t+1,i-1:i),vs(i-1,t+1)), -6);
end
end

need1 = [0 60 120 180 240 300]+1;
need2 = zeros(1,7);
need2_t1 = 1:2:13;
need2_t2 = 2:2:14;
need2_t = [0 1 51 101 151 201 223];
need2(need2_t1) = 2*need2_t+1;
need2(need2_t2) = 2*need2_t+2;
xxyy = xys(need2,need1);

vvv = vs(need2_t+1,need1);

```

附录 16

three_solve0.m 内容

```
clear

d_max = 55;

d_min = 0;

R0 = 450;

rs = zeros(1,224);

judge = 1;

while judge==1

while d_max - d_min>0.0001

d = (d_min+d_max)/2;

outs = cell(1,223);

bangs = zeros(100,100);

r0 = R0;

rs = zeros(1,224);

rs(1) = r0;

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/d-2*pi*r0/d)-81769;

r_limit = [r0 r0+d/2];

r1 = fzero(f,r_limit);

rs(2) = r1;

outs{1} = four_point_plus(rs(1:2),d);

for i = 3: 224

r0 = r1;

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/d-2*pi*r0/d)-27225;

r_limit = [r0 r0+d/2];

r1 = fzero(f,r_limit);

rs(i) = r1;
```

```

outs{i-1} = four_point_plus(rs(i-1:i),d);

end

for i1 = 1:100
    for j1 = i1+2:100
        in1 = outs{i1};
        in2 = outs{j1};
        bangs(i1,j1) = is_bang(in1,in2);
    end
end

if sum(sum(bangs))>0
    d_min = d;
else
    d_max = d;
end

end

d = d_max;

Rs = linspace(R0,R0+2*d,1000);

judge = 0;

for R = Rs
    outs = cell(1,223);
    bangs = zeros(100,100);
    r0 = R;
    rs = zeros(1,224);
    rs(1) = r0;
    f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/d-2*pi*r0/d)-81769;
    r_limit = [r0 r0+d/2];

```

```

r1 = fzero(f,r_limit);

rs(2) = r1;

outs{1} = four_point_plus(rs(1:2),d);

for i = 3: 224

r0 = r1;

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/d-2*pi*r0/d)-27225;

r_limit = [r0 r0+d/2];

r1 = fzero(f,r_limit);

rs(i) = r1;

outs{i-1} = four_point_plus(rs(i-1:i),d);

end

for i1 = 1:100

for j1 = i1+2:100

in1 = outs{i1};

in2 = outs{j1};

bangs(i1,j1) = is_bang(in1,in2);

end

end

if sum(sum(bangs))>0

disp(R)

R0 = R;

d_min = d;

d_max = 55;

judge =1;

break

end

```

```
end
```

```
end
```

```
d_solve = d_max;
```

附录 17

two_solve0.m 内容

```
clear
```

```
R0 = 880;
```

```
t0 = 1;
```

```
T = 1;
```

```
out_data = zeros(224,3);
```

```
fx = @(r) r.*cos(2*pi.*r/55);
```

```
fy = @(r) r.*sin(2*pi.*r/55);
```

```
while T>1/100000
```

```
for t = t0:T:500
```

```
outs = cell(1,223);
```

```
bangs = zeros(100,100);
```

```
r0 = hand_move(R0,t);
```

```
rs = zeros(1,224);
```

```
rs(1) = r0;
```

```
f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/55-2*pi*r0/55)-81769;
```

```
r_limit = [r0 r0+27.5];
```

```
r1 = fzero(f,r_limit);
```

```
rs(2) = r1;
```

```
outs{1} = four_point(rs(1:2));
```

```
for i = 3: 224
```

```

r0 = r1;

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/55-2*pi*r0/55)-27225;

r_limit = [r0 r0+27.5];

r1 = fzero(f,r_limit);

rs(i) = r1;

outs{i-1} = four_point(rs(i-1:i));

end

for i1 = 1:100

for j1 = i1+2:100

in1 = outs{i1};

in2 = outs{j1};

bangs(i1,j1) = is_bang(in1,in2);

end

end

if sum(sum(bangs))>0

disp(t);

t0 = t-10*T;

T = T*0.1;

break

end

end

end

t_solve = t - T;

r0 = hand_move(R0,t_solve);

rs(1) = r0;

```

```

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/55-2*pi*r0/55)-81769;

r_limit = [r0 r0+27.5];

r1 = fzero(f,r_limit);

rs(2) = r1;

for i = 3:224

r0 = r1;

f = @(r) r^2+r0^2-2*r*r0*cos(2*pi*r/55-2*pi*r0/55)-27225;

r_limit = [r0 r0+27.5];

r1 = fzero(f,r_limit);

rs(i) = r1;

end

out_data(:,1) = roundn(fx(rs)/100,-6)';

out_data(:,2) = roundn(fy(rs)/100,-6)';

out_data(1,3) = 1;

for i = 2:224

out_data(i,3) = roundn(next_v(rs(i-1:i),out_data(i-1,3)), -6);

end

need = [1 2 52 102 152 202 224];

ooo = out_data(need,:);

```